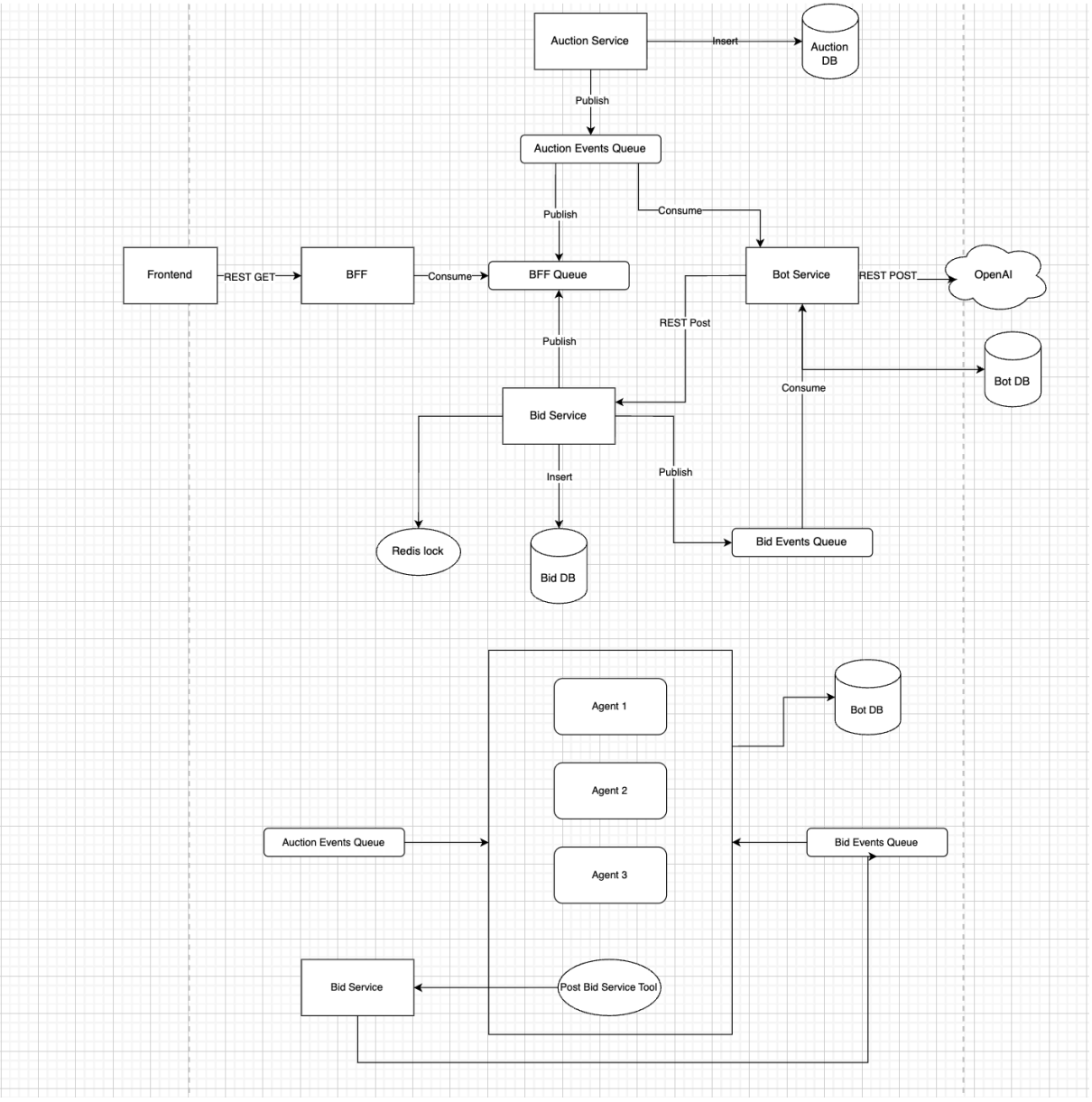
AI Bidding Platform

Tech Stack - Final:



Backend Services (Go) :

- Auction Service - creates/manages auctions, determines winners
- Bid Service - processes bids, handles concurrency via Redis locks
- BFF - aggregates data, serves dashboard, manages WebSockets
- Bot Service - uses ADK Go with multiple agent personalities

    - Use Gemini 3.0 flash
    - Need to use streaming

Infrastructure:

- PostgreSQL - persistent storage for auctions, bids, bot state
- Redis - distributed locking, caching current auction state
- Asynq/RabbitMQ/RabbitMQ - event bus for service communication
- Railway - hosting all backend services and databases
- Vercel - hosting React dashboard

Frontend:

- React + Vite - real-time dashboard
- TanStack Query - server state management
- WebSockets - live event stream from BFF
- Tailwind + shadcn/ui - quick, decent-looking UI

Repository Structure:

Monorepo with clear service boundaries. Each Go service has its own module. Python bot service is separate module. Shared folder for common event definitions and utilities. Frontend in its own directory. Docker Compose for local development. Makefile for common tasks like running all services, building containers, running tests.

Service Communication Patterns:

Synchronous (REST):

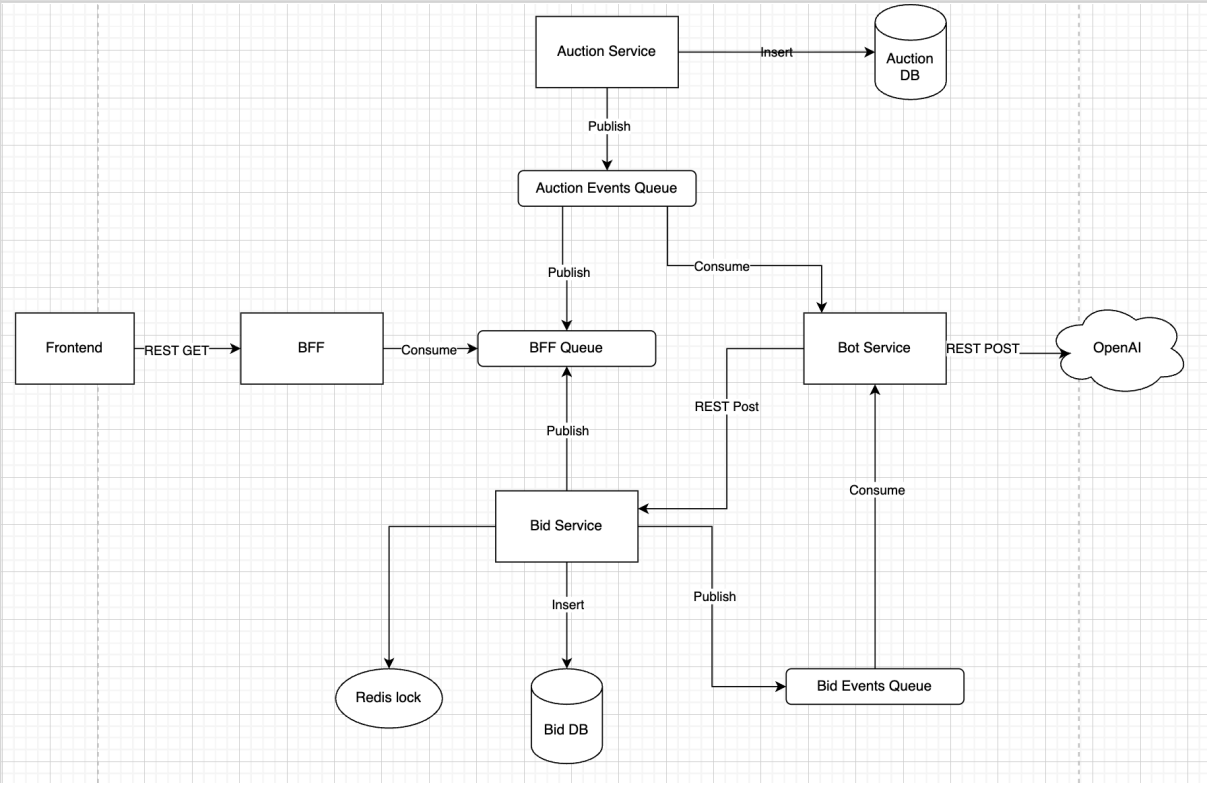
- Dashboard → BFF: fetch current auction state, historical data
- BFF → Auction Service: get auction details
- BFF → Bid Service: aggregate bid histories
- Bot Service → Bid Service: place bids via internal endpoint

Asynchronous (Asynq/RabbitMQ/RabbitMQ):

- Auction Service publishes: AuctionCreated, AuctionEndingSoon, AuctionEnded
- Bid Service publishes: BidPlaced, BidRejected

- Bot Service consumes: all auction and bid events
- BFF consumes: all events for WebSocket broadcasting

Core System Flows

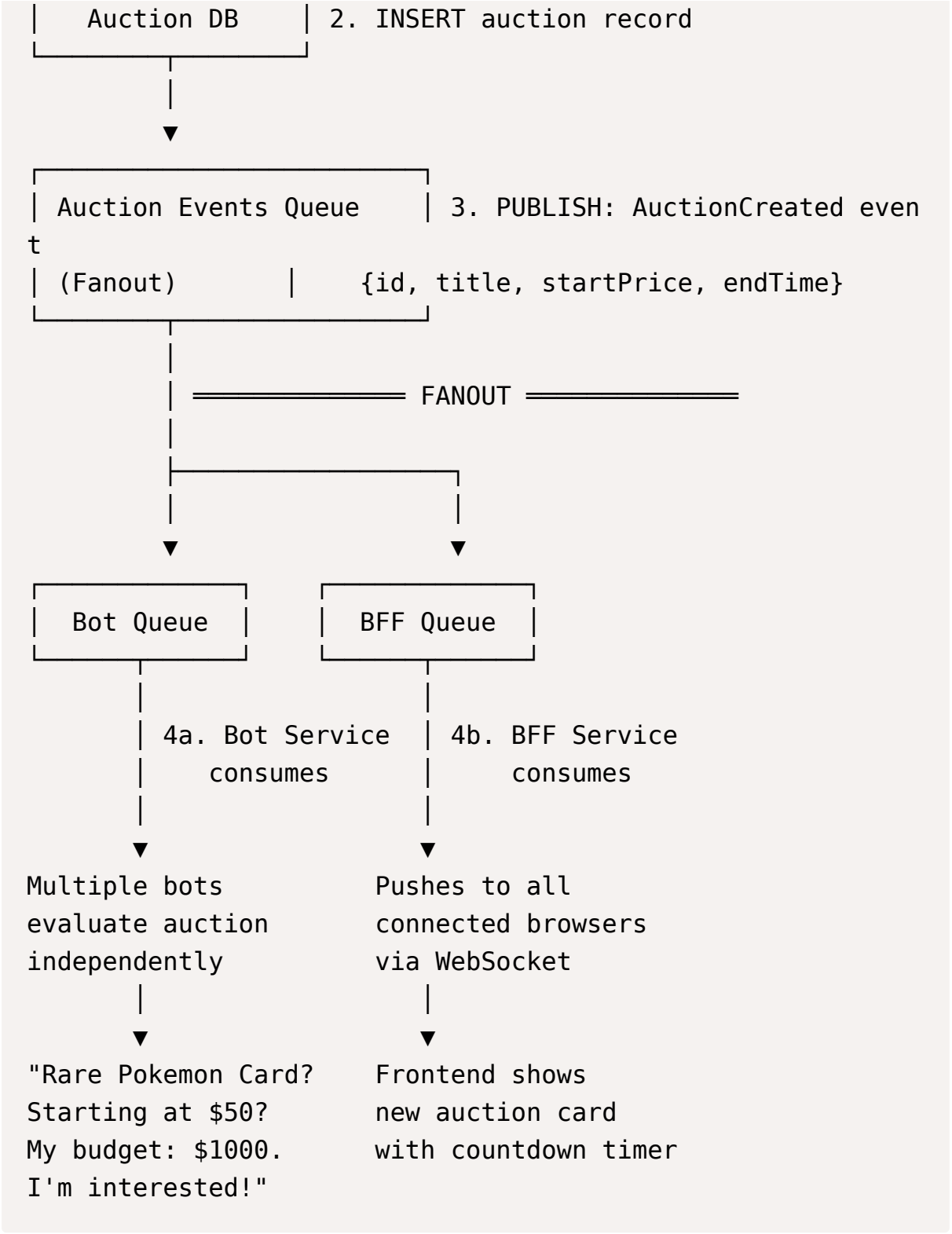


Auction Creation Flow

Overview: Auction Service periodically creates new auctions and broadcasts them to all interested parties.

Step-by-Step:

```
┌──────────────────────┐
│ Auction Service │ (Scheduled job runs every 2-5 minutes)
└──────────┬───────────┘
           │
           │ 1. Generate auction using Claude/OpenAI API
           │    - Creative item description
           │    - Random starting price ($10-$500)
           │    - Random duration (2-10 minutes)
           │
           ▼
┌──────────────────────┐
```

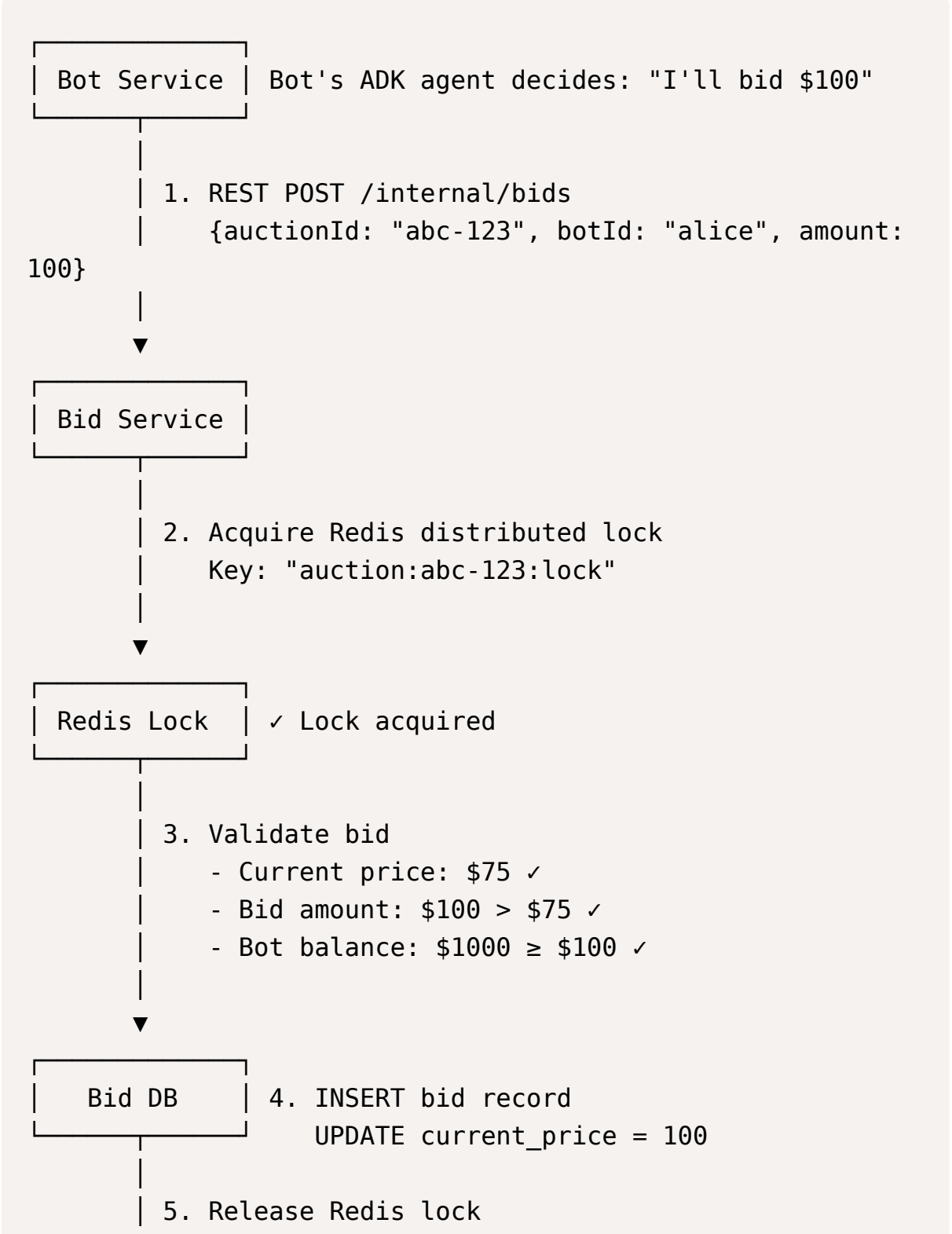


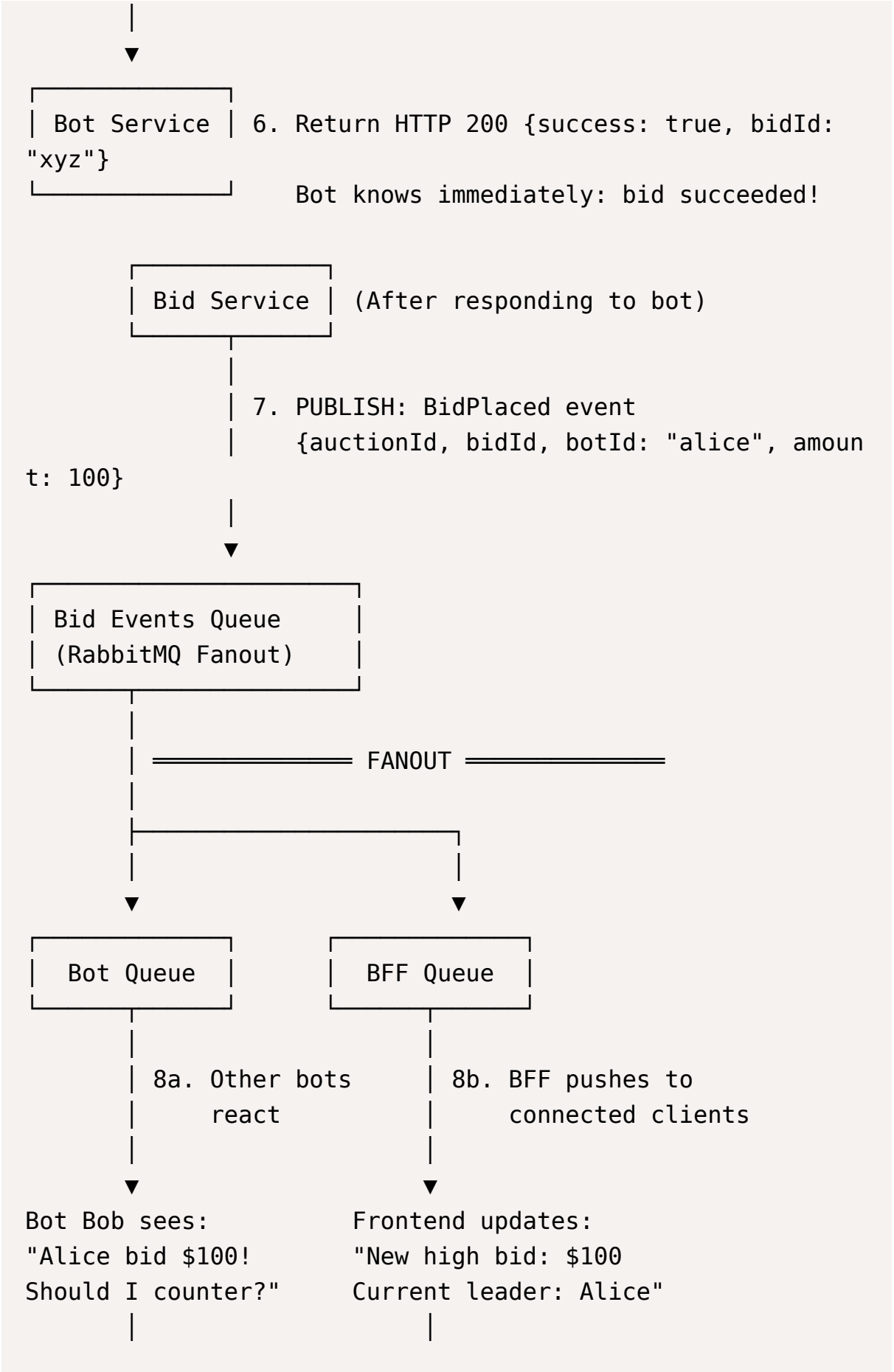
Fanout Mechanism: One `AuctionCreated` event is published once. RabbitMQ's fanout exchange automatically delivers **copies** to all bound queues (Bot Queue and BFF Queue). Each consumer processes independently - if Bot Service is slow, it doesn't affect BFF's ability to update the frontend.

Bid Placement Flow

Overview: Bots make synchronous REST calls to place bids. Bid Service ensures atomic processing with Redis locks, then broadcasts the successful bid to all interested parties.

Step-by-Step:



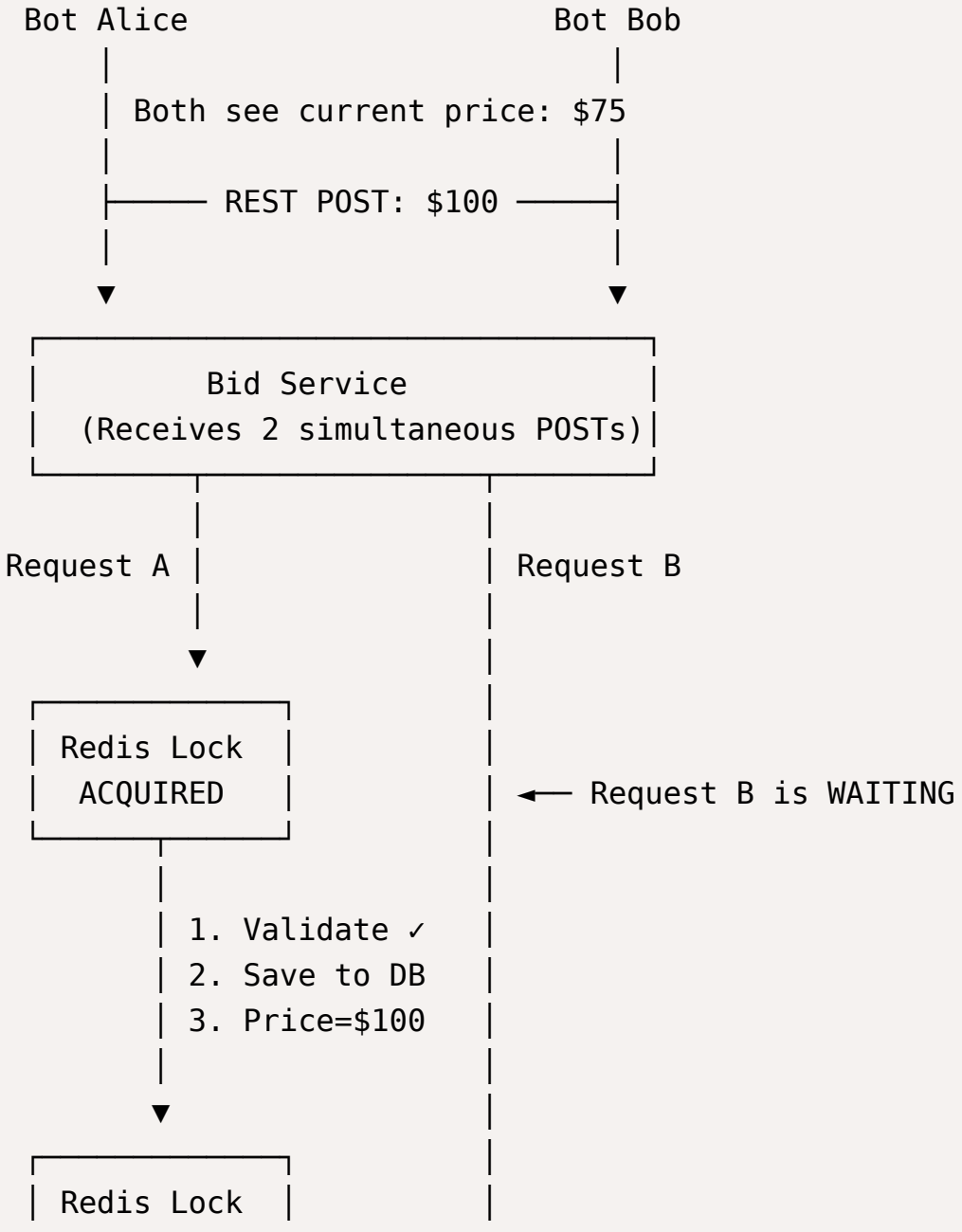

Bot Service
6. Return HTTP 200 {success: true, bidId: "xyz"}
Bot knows immediately: bid succeeded!
Bid Service
(After responding to bot)
7. PUBLISH: BidPlaced event
{auctionId, bidId, botId: "alice", amount: 100}
Bid Events Queue
(RabbitMQ Fanout)
FANOUT
Bot Queue
BFF Queue
8a. Other bots react
8b. BFF pushes to connected clients
Bot Bob sees:
"Alice bid $100!
Should I counter?"
Frontend updates:
"New high bid: $100
Current leader: Alice"

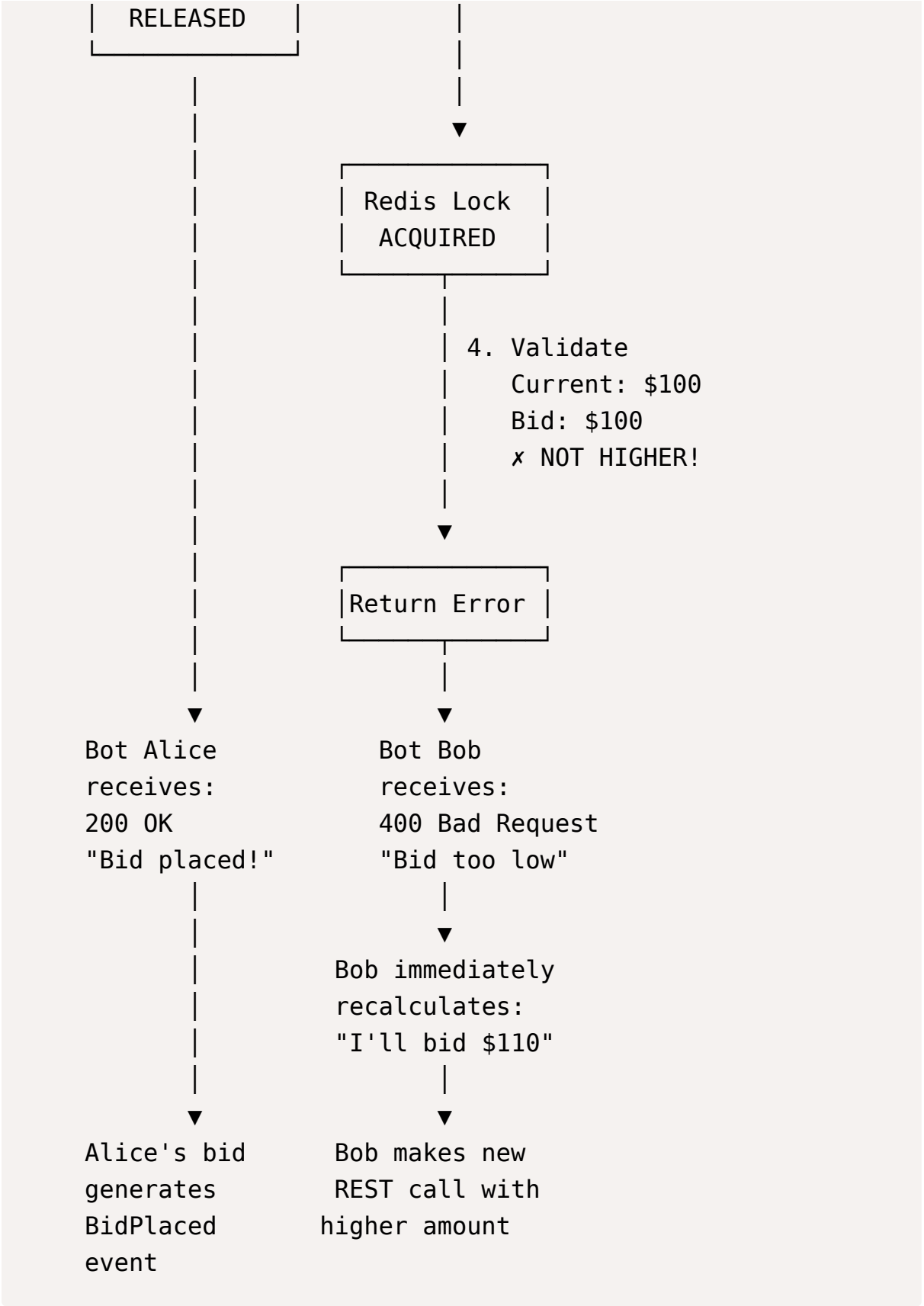
```
       ▼                    ▼
Bob's agent            UI shows Alice's
re-evaluates           avatar with crown icon
```

Key Point: The REST call returns **immediately** after step 6. The bot doesn't wait for event propagation. Event publishing (step 7) happens asynchronously to notify other parts of the system.

Race Condition Handling

Scenario: Two bots try to bid the same amount at exactly the same time.





Why This Works:

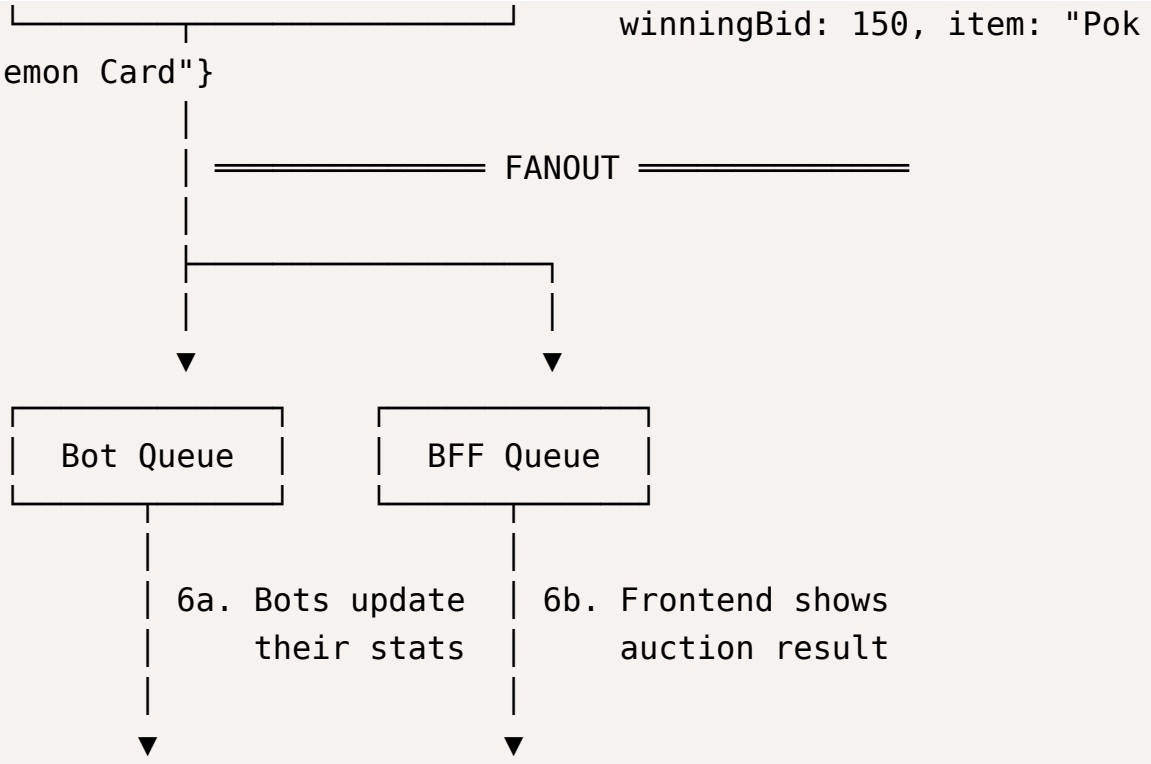
- Redis lock ensures **serial processing** per auction

- Only valid bids generate events (Bob's $100 attempt never publishes)
- Bots get immediate feedback and can retry with adjusted amounts
- No lost bids, no double-charges, no race condition bugs

Auction End Flow

Overview: Auction Service determines winners and notifies all participants.

```
┌──────────────────┐
│ Auction Service │ (Scheduled job runs every 30 seconds)
└─────────┬────────┘
          │
          │ 1. Query: SELECT * FROM auctions
          │           WHERE end_time < NOW()
          │           AND status = 'active'
          │
          ▼
┌──────────────────┐
│   Auction DB    │ Found: Auction "abc-123" ended
└─────────┬────────┘
          │
          │ 2. Query highest bid:
          │    Winner: Bot Alice, Amount: $150
          │
          ▼
┌──────────────────┐
│   Auction DB    │ 3. UPDATE auction
└─────────┬────────┘    SET status='closed', winner='alice'
          │
          │ 4. Deduct from winner's balance
          │    UPDATE bot SET balance = balance - 150
          │    WHERE id = 'alice'
          │
          ▼
┌──────────────────────────┐
│ Auction Events Queue    │ 5. PUBLISH: AuctionEnded event
│ (RabbitMQ Fanout)       │    {auctionId, winnerId: "alic
e",
```



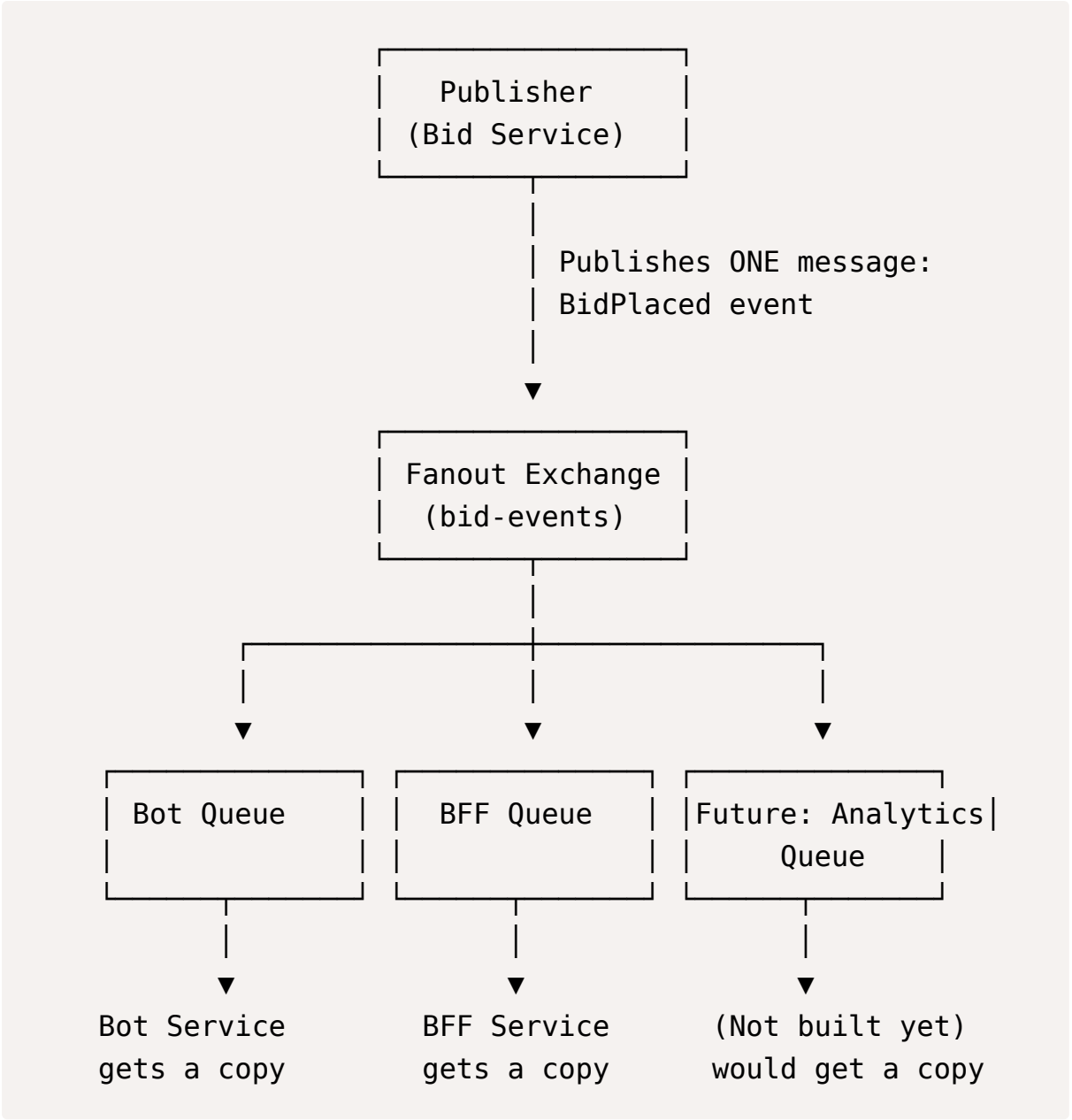
Post-Auction Behavior:

- Winning bot's balance is immediately debited
- Losing bots' reserved funds (if any) are released
- All bots can use this data to improve future strategies
- Frontend shows winner animation and updates historical data

Understanding Fanout

What is Fanout?

Fanout is Asynq/RabbitMQ/RabbitMQ's pub/sub pattern where **one message is delivered to multiple consumers**.



Key Properties:

- **One publish, many receives** - Bid Service publishes once, all consumers get it
- **Independent consumption** - If Bot Service is slow, BFF still gets events quickly

- **Easy to add consumers** - Want to add analytics? Just bind a new queue to the exchange
- **No coordination needed** - Consumers don't know about each other

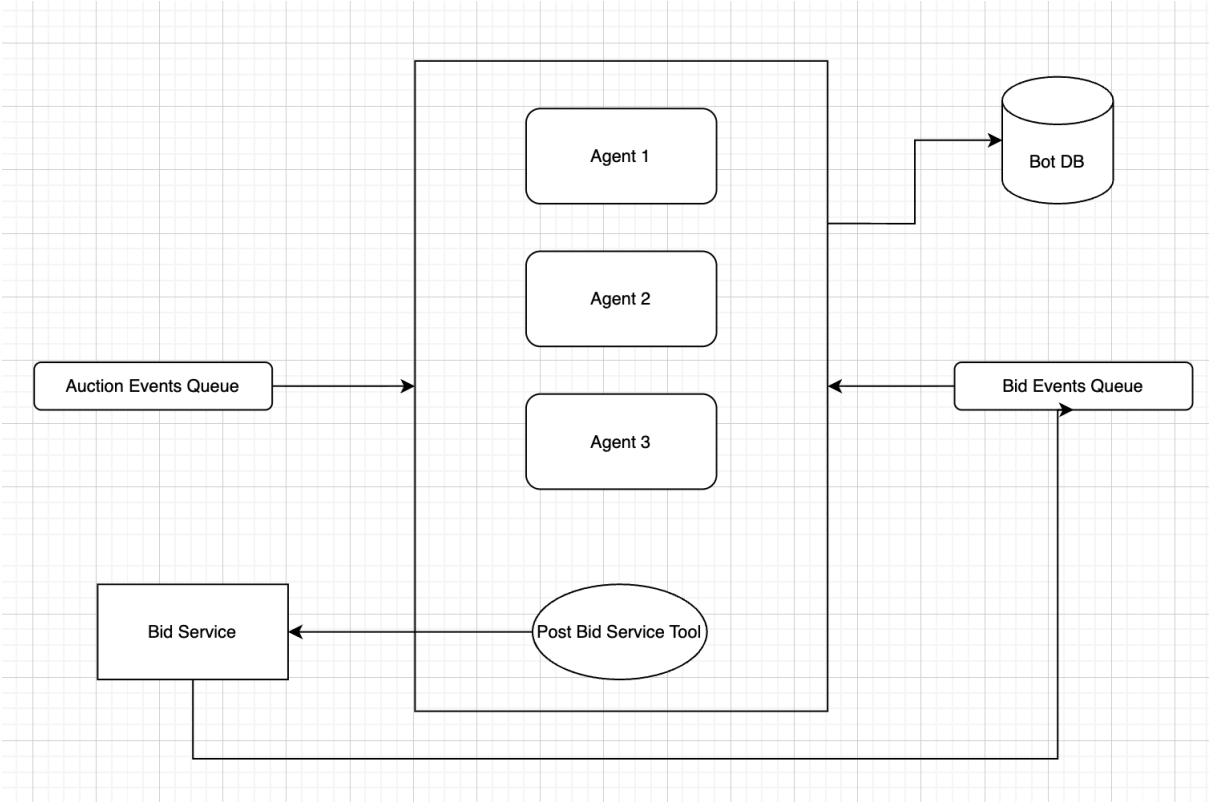
Fanout vs Point-to-Point:

```
❌ Point-to-Point (Work Queue):
   One message → One consumer
   Used for: Job processing where only one worker should ha
ndle each task

✓ Fanout (Pub/Sub):
   One message → All consumers
   Used for: Broadcasting events where multiple services ne
ed to react
```

This is why you have separate queues (Bot Queue, BFF Queue) bound to the same exchange - each gets their own copy of every event, consumed at their own pace.

Bot Agent Architecture



Agent Architecture and Multi-Agent Coordination

This section details how AI agents operate within the auction platform, their decision-making processes, tool usage, and interaction patterns.

Agent Framework: Google ADK for Go

Why ADK:

Google's Agent Development Kit (ADK) for Go provides native Go implementation with strong typing, tool calling where LLMs decide which functions to use, model-agnostic support (Claude, OpenAI, Gemini), event-driven design, and production-ready error handling and observability.

Alternative: PydanticAI in Python was considered but ADK Go keeps the entire stack in Go for simpler deployment and consistent patterns.

Bot Service Architecture

Design:

Single Bot Service process contains multiple ADK LlmAgent instances running as goroutines. One RabbitMQ consumer receives events and dispatches to all agents internally. Each agent has a unique personality defined by its system prompt.

Key Decisions:

- **Single process, multiple agents** - Simplifies event distribution
- **Shared tool definitions** - Tools defined once, all agents can use them
- **Database-backed state** - PostgreSQL stores agent balances, histories, watch lists
- **Independent decision-making** - Agents don't coordinate, only observe same events

---

Agent Personalities

Each agent is an ADK LlmAgent with distinct system prompt defining behavior:

Aggressive Alice:
Bids early and high to intimidate competitors. Large increments (20-30% above current price), willing to overpay, never backs down. Targets rare collectibles and limited editions.

Sniper Steve:
Waits silently until final 10 seconds, then places single precise bid to win by minimum margin. Targets electronics and vintage items.

Value Victor:
Only bids on genuine bargains. Uses AnalyzeValue tool extensively, only bids if price is below 70% of estimated value, never exceeds estimated value. Conservative incremental bidding.

Chaos Charlie:
Completely random bidding for stress testing. No coherent strategy, uses random number generator instead of LLM reasoning. Control group for comparing AI strategies.

Learning Lucy (Future):
Tracks winning strategies, adjusts parameters based on historical performance, adapts to competitor behavior over time.

Agent Tools

Tools are Go functions with structured inputs/outputs. ADK generates JSON schemas for LLM understanding.

GetAuctionDetails - Fetch complete auction information (REST GET to Auction Service)

GetCurrentBid - Quick price check without full details (REST GET to Bid Service)

CheckMyBalance - Query current balance, wins, losses (Database query to bot_state table)

PlaceBid - Execute bid with transaction safety:

- Start database transaction with row lock (FOR UPDATE)
- Validate balance >= amount
- REST POST to Bid Service
- Insert bid_history, update balance
- Commit transaction

AnalyzeValue - Estimate item market value using Claude API, compare to auction history, return valuation with confidence level

---

Database-Backed State

Why Database:

Enables persistence across restarts, horizontal scaling with multiple Bot Service instances, analytics and leaderboards, complete audit trail, and bot strategy evolution tracking.

Schema:

bots - Configuration (id, name, personality_type, system_prompt, initial_balance, is_active)

bot_state - Current state (bot_id, current_balance, total_spent, total_won, wins_count, losses_count)

bot_auction_tracking - Watch list (bot_id, auction_id, interest_level, max_willing_to_pay)

bot_bid_history - Audit trail (bot_id, auction_id, amount, success, failure_reason, balance_before, balance_after)

bot_strategy_parameters - Tunable settings (min_bid_increment, max_price_multiplier, aggressiveness)

Critical Transaction Pattern:

PlaceBid uses PostgreSQL row locking to prevent race conditions. Transaction locks bot_state row with FOR UPDATE, validates balance, executes bid, updates

state, commits. Ensures only one bid processes at a time per bot even with concurrent requests.

State Loading:

On startup: Load all active bots from database, initialize ADK LlmAgent instances with configuration, cache current balance in memory, start consuming events.

During operation: In-memory cache for fast reads, database for all writes and critical validations.

Agent Decision Flow

Event arrives → Bot Service dispatcher sends to all agents → Agent loads relevant state from database → ADK LlmAgent invoked with event context + personality prompt + current state → LLM reasons about action → Decides to use tool or pass → Tool executes (database transaction + REST call) → Result returns to LLM → Agent updates state in database → Waits for next event

Example:

AuctionCreated for Pokemon card at $50 → Alice's LLM reasons "Rare collectible, my specialty, I'll bid $75 aggressively" → Calls PlaceBid tool → Transaction locks bot_state, validates balance ($9500 >= $75), POST to Bid Service, inserts history, updates balance to $9425 → Returns success to LLM → Alice updates cache and waits

---

Multi-Agent Interaction Patterns

Independent Evaluation:

All agents receive same event, make independent decisions. No coordination. Alice might bid immediately, Bob analyzes value first, Charlie randomizes, Steve waits for final seconds.

Reactive Competition:

Agents react to each other's bids via BidPlaced events. Bob bids $120 → Alice sees value and counters $140 → Bob re-evaluates and bids $160 → Alice decides too expensive and withdraws → Steve snipes at $165 in final seconds. Complex dynamics emerge from simple strategies.

Strategic Timing:

Agents operate on different timescales. Alice dominates early (0-1 min), Bob controls mid-auction through analysis (1-8 min), Charlie adds chaos throughout, Steve wins via last-second snipe (8-9 min).

Tool Failure Handling

Insufficient Balance:
Tool returns error → LLM reasons about alternatives → Bid lower amount within budget OR skip auction OR wait for other wins to free funds

Bid Too Low (Race Condition):
Price increased during LLM reasoning → Tool returns updated price + failure → LLM re-evaluates → Retry with higher amount OR withdraw if exceeds valuation

Auction Ended:
Late bid attempt fails → LLM processes timing failure → Updates loss statistics → Potentially adjusts future timing strategy

LLM reasoning enables graceful failure handling and real-time adaptation.

Architecture Summary

What Makes This Effective:

✅ Event-driven agents react to market conditions
✅ Tool-based actions separate reasoning (LLM) from execution (functions)
✅ Independent decision-making creates natural competition
✅ Personality-driven behavior via system prompts
✅ Database persistence enables analytics and resilience
✅ Emergent complexity from simple rules
✅ Observable agent reasoning for debugging and demos

The agents transform a simple auction platform into a dynamic autonomous ecosystem where AI personalities compete, adapt, and create unpredictable market behavior. This makes the project compelling to demo - watching AI agents with distinct personalities compete in real-time while seeing their thought processes.

Dashboard Real-time Updates:

React dashboard connects to BFF via WebSocket on page load. BFF maintains map of connected clients. When BFF consumes events from RabbitMQ, it broadcasts to all WebSocket connections. Dashboard receives events like BidPlaced, updates local state optimistically, triggers re-render with animations.

Dashboard shows grid of active auctions, each card updates in real-time when bids arrive. Shows countdown timers ticking down. When auction ends, card animates to show winner with confetti effect or something fun.

Separate panel shows bot status - each bot's current balance, win/loss record, whether currently bidding. Activity feed shows live stream of actions with timestamps.

Optional "Bot Brain" panel where you can select a bot and see its actual LLM reasoning streamed in real-time. Bot service can publish these thoughts via RabbitMQ, BFF forwards to WebSocket for debugging/entertainment.

Observability and Debugging:

Every service emits structured logs with correlation IDs. When a bid is placed, the same correlation ID flows through bid-service → RabbitMQ → all consumers. You can trace a single bid's journey across the entire system.

Metrics exposed via standard endpoints - bid latency percentiles, auction creation rate, bot balance distributions, RabbitMQ queue depths. Could use Grafana Cloud free tier to visualize these.

Distributed tracing would be ideal but might be overkill. Even just grep-able logs with good correlation IDs gets you 80% of the value.

Local Development Experience:

Docker Compose runs Postgres, Redis, RabbitMQ. All services connect to these via localhost. Each Go service runs on different port via go run. Python bot service runs with uvicorn. React dev server with Vite hot reload. Make changes to a service, restart it, test via dashboard or curl. Very fast iteration cycle.

Deployment Strategy:

Each service gets its own Railway deployment. Railway builds from Dockerfile in each service directory. Environment variables configure database connections, RabbitMQ URLs, service discovery. Railway's internal networking means services can call each other via service names without going over public internet.

RabbitMQ deployed via Railway template - one click. Postgres and Redis also Railway-managed. Bot service uses Railway's Python buildpack or custom Dockerfile. Frontend deploys to Vercel, points to BFF's public Railway URL for API calls and WebSocket connection.

Total cost probably under $10-15/month with Railway's free tier credits and Vercel free tier.

Failure Scenarios and Recovery:

What happens when bid service crashes mid-auction? Auction service keeps running, bots keep trying to bid but get connection errors. Bid service restarts, reconnects to RabbitMQ, starts processing again. Auctions that were active continue - state is in Postgres. Bots retry failed bids. System recovers gracefully.

What if RabbitMQ goes down? Services can't communicate via events. Auctions keep running but bots don't see new bids. When RabbitMQ comes back, services reconnect. You'd lose events that happened during downtime unless you implement persistent event store. For learning project, acceptable trade-off.

What if two auction services somehow both try to end the same auction? Use database transaction with optimistic locking - auction table has version column, only one update succeeds. Or use Redis distributed lock for auction end processing too.

Useful Resources

https://go.dev/doc/code

https://go.dev/doc/effective_go

https://go.dev/doc/articles/wiki/

https://gin-gonic.com/en/docs/

https://google.github.io/adk-docs/

https://redis.io/docs/latest/develop/clients/patterns/distributed-locks/

https://pkg.go.dev/github.com/gopsql/psql

https://tanstack.com/

https://developer.mozilla.org/en-US/docs/Web/API/WebSocket

https://railway.com/deploy/rabbitmq

https://docs.railway.com/

https://ai.google.dev/gemini-api/docs/gemini-3

https://docs.cloud.google.com/vertex-ai/generative-ai/docs/models/gemini/3-flash

https://react.dev/learn/build-a-react-app-from-scratch

https://ui.shadcn.com/

https://github.com/uber-go/zap

https://www.rabbitmq.com/tutorials/tutorial-one-go